



G-Maker：從零打造全自動化 AI 短影音生成 workflow 與本地部署實戰

這是什麼&&我學到了什麼

這是我為了探索 AI 媒體生成應用而開發的「全自動化短影音生成系統」。這套系統能將任何純文字腳本或演講逐字稿，全自動轉換為帶有 AI 語音配音與動態字幕的短影音；畫面部分則完全交由語言模型撰寫提示詞（Prompt），並接入 text-to-Video (T2V) 影片生成模型即時運算生成，最後自動拼接完成。

在這個專案中，我最大的技術突破在於**工作流（Workflow）的模組化設計與實裝**。我不僅學會了利用 Python 的 `uv` 工具高效管理環境與套件，更深入應用了 SQLite 的狀態管理機制與 FFmpeg 的影音自動化剪輯。另外，我成功在本地端部署了 Wan 2.2 這類重型文字生成影片模型，並透過自建 API 串接至系統中，大幅累積了系統架構設計與邊緣運算的實作經驗。然後最重要的是，**我學會了如何最大化的利用手邊的資源來突破技術限制**，比如借用同學的高性能顯卡、聯絡 elevenlabs 開發者爭取免費試用額度、導入社群優化的量化模型工作流來提升算圖效率、利用 github education 送學生的免費 microsoft azure 額度來串接雲端語言模型 ai 服務等等。

動機與開發起點

近年很多人沉迷於短影音，我有時也會滑到忘記時間，有天我發現很多科普類或是內容導向的影片很適合用 ai 來自動化生產且剛好那陣子 ai 影片模型很流行，於是基於對軟體自動化與 AI 生成技術的好奇，我決定自己打造一套端到端的影音生成專案。

起初，我利用 github education 免費的 Azure 額度串接 Whisper 與 GPT-4o mini 進行基礎的配音與字幕處理。我的「第一版系統」其實非常陽春：畫面僅依賴 ai 圖片生成與簡單字幕。然而，當我首度嘗試接入 Azure 的 Sora 模型來生成動態影片時，昂貴的 API 成本讓我第一次測試就把免費額度直接燃燒殆盡。這個痛點迫使我轉換方向，毅然決然投入開源 AI 影片模型的本地部署研究。

本地部署的挑戰與硬體效能優化

在嘗試雲端影片 ai API 失敗後，我決定挑戰將開源影片模型 **Wan 2.2** 部署於地端。這是我第一次處理需要極端 GPU 算力的深度學習模型，因此我借用了同學配備 16GB 顯卡的高性能電腦，將其作為算力節點架設 API 供系統調用。

一開始的光景可說是一場災難。我遭遇了無止盡的環境建置地獄：從 PyTorch 衝突、NVIDIA Toolkit 與 CUDA 不匹配，一路到令人崩潰的 flash_attn 編譯錯誤。且即便環境裝好，受限於 16GB VRAM，我也只能勉強跑得動參數最低的模型，產出的畫面時常扭曲崩壞，且算圖效率極低，生成短短 5 秒的影片片段竟需要耗費高達 5 分鐘。

為了解決資源瓶頸與品質問題，我開始深入研究模型優化技術。我捨棄了官方基礎腳本，改導入 **ComfyUI 與社群高度優化的量化模型工作流**；接著在後端利用 Python 串接comfyui API。這項改動是巨大的轉捩點：它大幅壓縮了顯存占用，讓我終於能在那張 16GB 顯卡上順利使用更高參數版本的影片模型，明顯提升了影片品質。**這項實戰讓我深刻體會到，懂得如何進行硬體資源調度和開源社區的資源來突破極限，與寫出漂亮的程式邏輯一樣重要。**

系統架構與工作流自動化設計

這專案最讓我有成就感的是其「7 階段自動化工作流」的架構設計。在開發初期，我很快意識到：假如只是寫個腳本依序把 API 呼叫一遍，一旦其中某個環節（例如網路斷線或算圖出錯）失敗，整個流程就會崩潰，導致需要重新啟動並消耗不必要的 API 成本與算力。

因此，我導入了狀態管理機制。我利用 SQLite 建立了一個任務追蹤資料庫，記錄每個影片專案的生命週期與處理狀態。

我的自動化工作流包含了：

1. **AI 寫影片腳本 (Script Generation)**：這是系統運作的起點。我設計了一套多層次的 LLM 提示詞工程，能將長文或演講逐字稿自動清洗成純素材（去除廢話與口語），再依據短影音（TikTok / YouTube Shorts）的演算法特性，重新潤飾成具備「強烈破題鉤子 (Hook)」與「行動呼籲 (CTA)」、節奏明快的專業腳本。
2. **TTS 語音生成**：接收優化後的腳本，先處理長文本的切割(避免檔案太大)，再透過我參加 2025 devjam 工作坊認識的elevenlabs開發者免費要到的試用會員，調用elevenlabs API 產出逼真人聲。
3. **精準動態字幕**：串接azure Whisper 生成逐字稿，這部分我特別將一般 SRT 字幕格式轉換為 ASS 字幕格式，實現動態字幕(字母級的跳動或是變色之類的)

4. **視覺提示詞生成 (Prompt Generation)**: 為了引導影片模型產出穩定高品質的畫面，我設計了 `Subject + Scene + Motion + Aesthetic` (主體+場景+動態+美學) 的結構化 Prompt 公式。LLM 會深入分析腳本分鏡與音訊時間軸，判斷需要畫面切換的時機，並用 structured output 生成 AI 影片生成需要的畫面描述跟時間參數。
5. **AI 影片生成 (Prompt-to-Video)**: 將視覺提示詞發送給本地 ComfyUI API 工作流，調用量化過後的 Wan 2.2 開源影片模型進行本機推論算圖，產出切合文意的畫面。
6. **多軌影音組裝**: 封裝 FFmpeg 的複雜指令，進行圖層疊加、背景模糊與時軌對齊。此外，在後期設計中，我加入了一個自己製作、僅有「開關嘴兩張圖片」的貓咪虛擬主播融入工作流中，專門用來填充沒有 AI 生成影片時的畫面空白。(節省運算時間)
7. **最終渲染**: 燒錄特效字幕並鎖定 60 FPS 輸出影片。

心得與反思

這次的開發經驗大幅拓寬了我對軟體工程的視野。以前我總習慣專注於單一功能的實作，但這次為了確保整套自動化工作流的穩定性，我不得不從零開始學習用 SQLite 處理任務的中斷恢復機制，甚至得在本地端親自架設與維護模型 API。這讓我深刻體會到，系統架構設計與環境部署也是實際開發中不可或缺的一環。

過程中，處理環境衝突與硬體效能調校讓我遇到了不少挫折，加上目前的開源影像生成模型偶爾還是會產出不穩定的畫面，且運算非常耗時。然而，當我真的看著這套全自動化系統順利跑完流程，把一段純文字轉換成具有聲光效果的完整短影音時，那份成就感是非常真實的。後來，我也將系統自動生成的影片上傳到 YouTube 進行測試，最終累積了五萬多次的觀看。這雖然稱不上是龐大的數字，但能看著自己親手寫出的程式產出具備實際價值的內容，仍讓我感到十分開心。

此外，這次專案也大幅提升了我的尋找與整合資源的能力。在硬體資源與預算有限的情況下，我學會了向同學借用高階顯卡來搭建算力節點、在開源社群中尋找並導入優化方案，以及積極向開發者爭取免費的 API 額度來突破技術瓶頸。這次的實戰經驗不僅證明了我具備從無到有、獨立完成並除錯全端專案的能力，也讓我有信心在未來繼續優化這套系統，例如嘗試導入 Agentic 架構，讓 AI 能根據內容自主完成更彈性的剪輯調度。

成果影片

初版圖片生成版本: <https://www.youtube.com/shorts/bdb3T3eim5k>

2版加入低參數影片模型和講話貓貓: <https://www.youtube.com/shorts/s5U5yxZ2D-8>

最終版本加入高參數影片模型和動態字幕和背景模糊效果: <https://www.youtube.com/>

[shorts/BnTLk49_uZE](#)

參考資料

我的 GitHub 專案倉庫: https://github.com/Kelsier64/g_maker

依賴管理工具 uv: <https://github.com/astral-sh/uv>

開源影片生成模型 (wan2.2): <https://github.com/Wan-Video/Wan2.2>

ComfyUI: <https://github.com/comfyanonymous/ComfyUI>